



ETH AI CENTER

Course Project

Project Details and Ideas



Course Project Gipfelsturm

Gipfelsturm (German for "summit attempt") is an open-ended engineering challenge. Train a language model on up to 64 GPU hours. It's up to you to decide if you run on 16 nodes for 1h or on one node for 64 hours while you push for the best results you can achieve. <https://github.com/swiss-ai/Isaie-ss26-gipfelsturm>

There is no single correct answer. It's about making informed engineering decisions, run test/experiments, and learn from what works.

Setup

The repository below uses Megatron-LM, the industry-standard framework for distributed LLM training.

The dataset we prepared for comparisons is Nemotron-ClimbMix, a curated pre-training corpus already converted to Megatron's binary format and stored on capstor.

The tokeniser is GPT-2 BPE (50,257 vocab size) which allows direct comparison with nanoGPT baselines.

Course Project Gipfelsturm

There are two dimensions on which you can improve. Your team should focus on at least one.

Challenge one: improve loss or compare with alternative method given fixed time

Compute the empirical throughput of your model and batch size. Then compute the number of tokens you can process in theory in a 30min, 1h, and 2h time window. This defines the number of steps your model/batchsize combination should be trained. Model size, learning rate, schedule, batch size, and training recipe are all your choice. The only constraint is the clock on the wall. Aim for lowest eval loss.

Theoretical Time	GPU-hours	Natural Model Scale
30min	16	135M-3B
1h	32	3B-8B
2h	64	8B+

Course Project Gipfelsturm

There are two dimensions on which you can improve. Your team should focus on at least one.

Challenge two: maximum throughput

Achieve the highest tokens/sec/GPU for a given model size on up to 8 nodes/32 GPUs.

Model-Parallelism	Model Size	Systems Challenge
Single-GPU	~8B	No model parallelism (TP=1, PP=1)
Single-Node	~32B	Intra-node tensor parallelism (TP=4)
Multi-Node	~140B	Pipeline parallelism across nodes (TP=4, PP=4)

Course Project Gipfelsturm

Baselines: We'll provide some baselines over the next days.

How to work: Start small and proceed systematic. Do not submit larger runs if you do not have a clear hypothesis. Throughput runs only need a dozen steps to be accurate.

Teams: Size of 3-4 students. Coordinate to split the work and compute.

Deliverable 1: Find a team and **submit ~1 page proposal by 14th of April** which lists the team members and roughly describes the main direction of your project **per student**, related work, milestones, etc.

Deliverable 2: Final write-up, codebase, W&B runs as a zip file through moodle by **29th of May**. Every submitted report requires motivation, method/contribution, and an evaluation. Evaluation should contain the necessary artifacts (e.g. scalability plot, NSYS traces, throughput evaluations, loss curves, other tests, etc).

Award: Exemplary submissions may be awarded a special **Letter of Distinction** by the course lecturers.

Possible Project Features

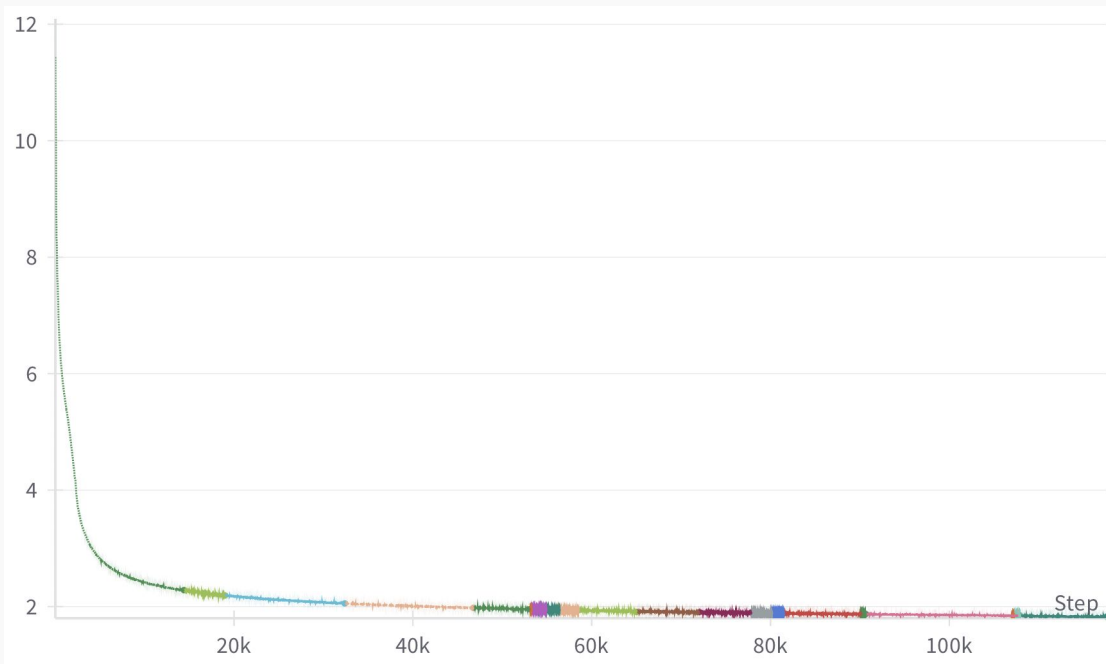
Large scale runs spans across several weeks/months

It's **crucial** to checkpoint properly the states of the run

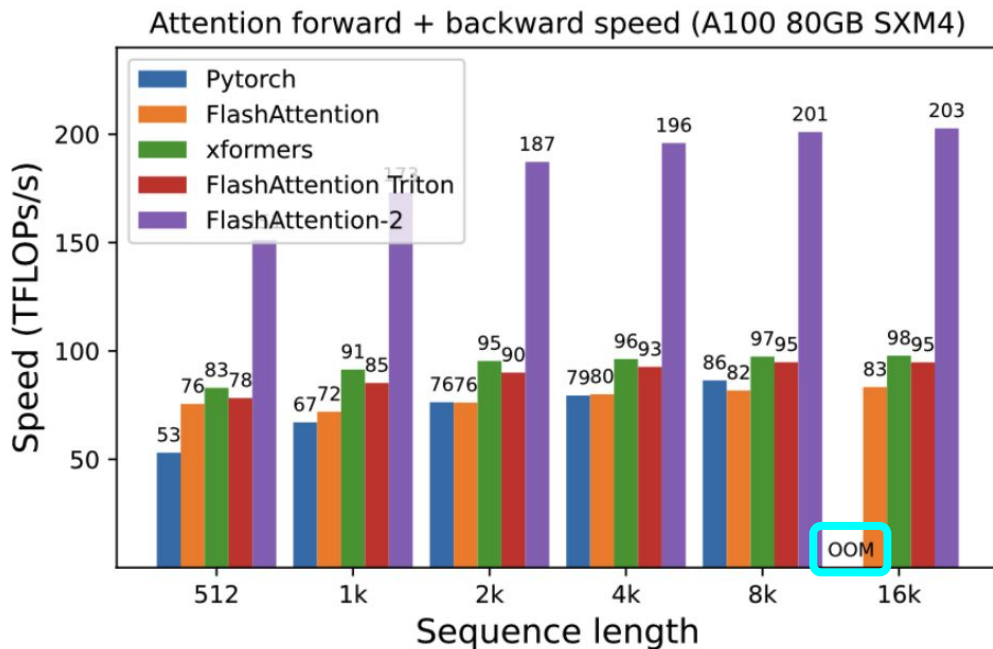
 Craft an auto job resubmission mechanism

 Craft a graceful exit mechanism that stores a checkpoint upon hitting the job time limit

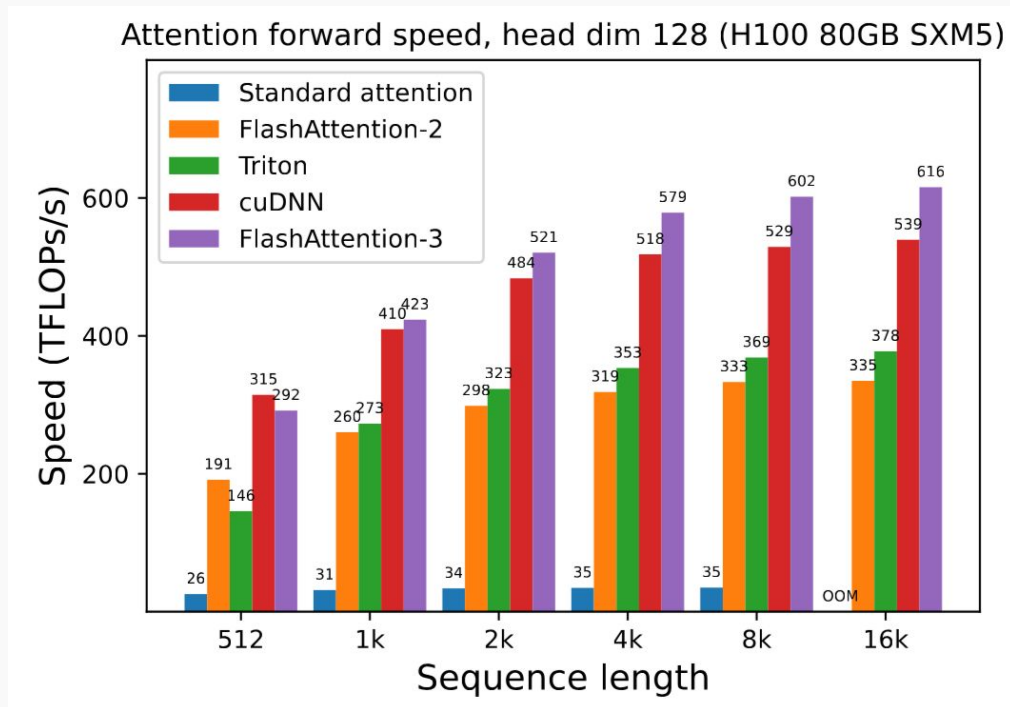
 Enable [Torch Fault Tolerance](#) (e.g. DiLoco for cross datacenter training)



Possible Project Features



Possible Project Features



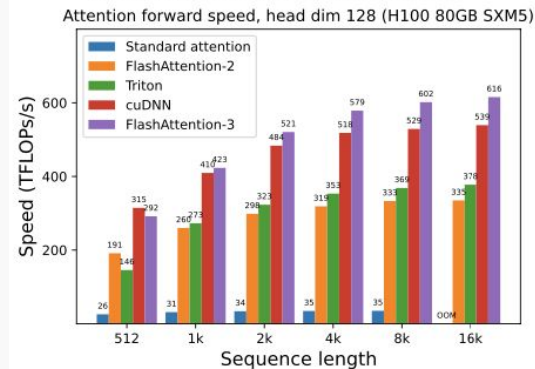
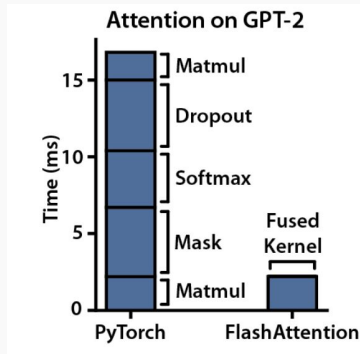
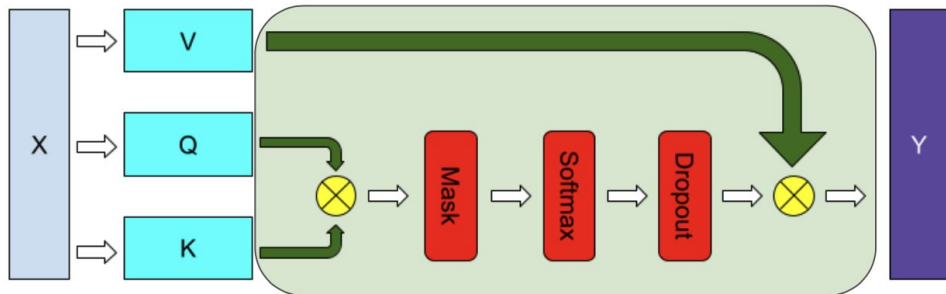
Possible Project Features

Currently using Torch's
scaled_dot_product_attention

 Leverage FlashAttentionV3

 Leverage CUDNN SDPA Backend

 Leverage [CUDNN-FrontEnd API](#)



Possible Project Features

More kernel fusion ⚡

 Leverage torch.compile

 Leverage CUDA Graphs

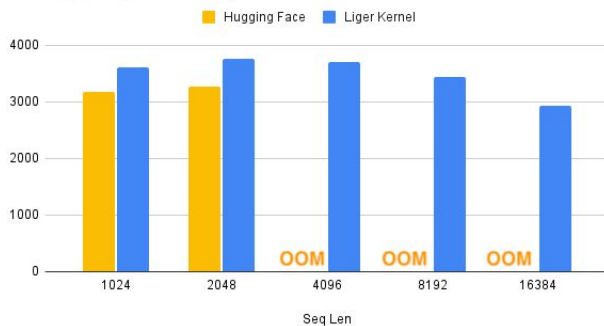
 Leverage [Liger-Kernel](#)

 Leverage [Cut Cross-Entropy](#)

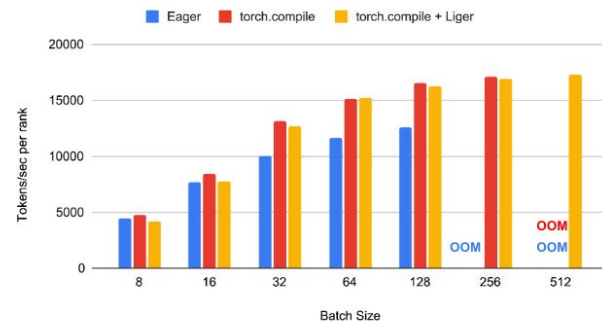
 Leverage [Quack kernels](#)

 Leverage CUDA/CUTLASS

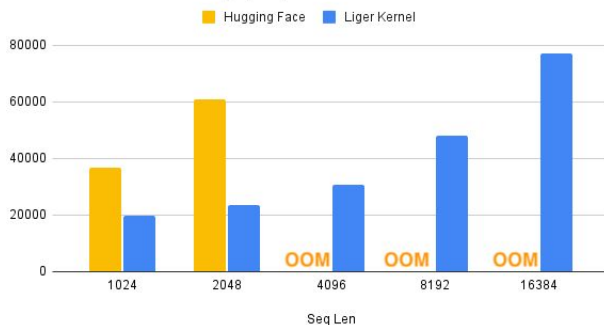
Throughput (tokens/sec)



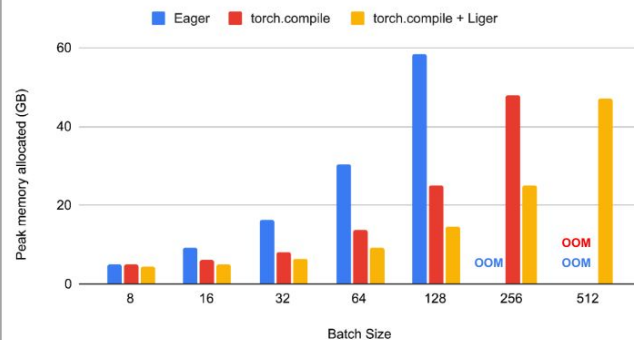
Tokens/sec per rank vs Batch Size



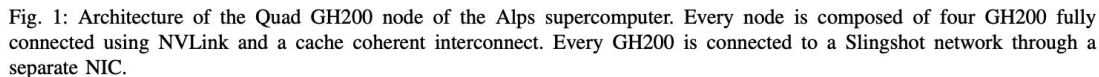
Peak Reserved Memory (MB)



Peak memory allocated (GB) vs Batch Size



 Leverage CPU Activation offloading w/ [TransformerEngine](#) (very useful for the GH200!)



Possible Project Features

Let's burn more GPUs 🔥



Enable Torch's DDP

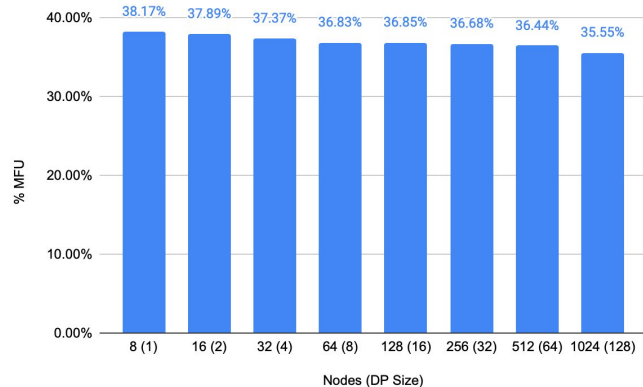


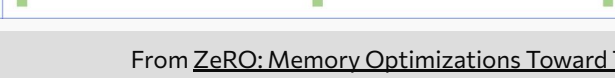
Enable Torch's FSDPv2



Enable DeepSpeed

ZeRO-1/2/3



	Parameters	Gradients	Optimizer States	Memory Consumed	K=12 $\Psi=7.5B$ $N_d=64$
Baseline				$(2 + 2 + K) * \Psi$	120GB
P_{os}				$2\Psi + 2\Psi + \frac{K * \Psi}{N_d}$	31.4GB
P_{os+g}				$2\Psi + \frac{(2 + K) * \Psi}{N_d}$	16.6GB
P_{os+g+p}				$\frac{(2 + 2 + K) * \Psi}{N_d}$	1.9GB

Possible Project Features

Enable long-context training ...

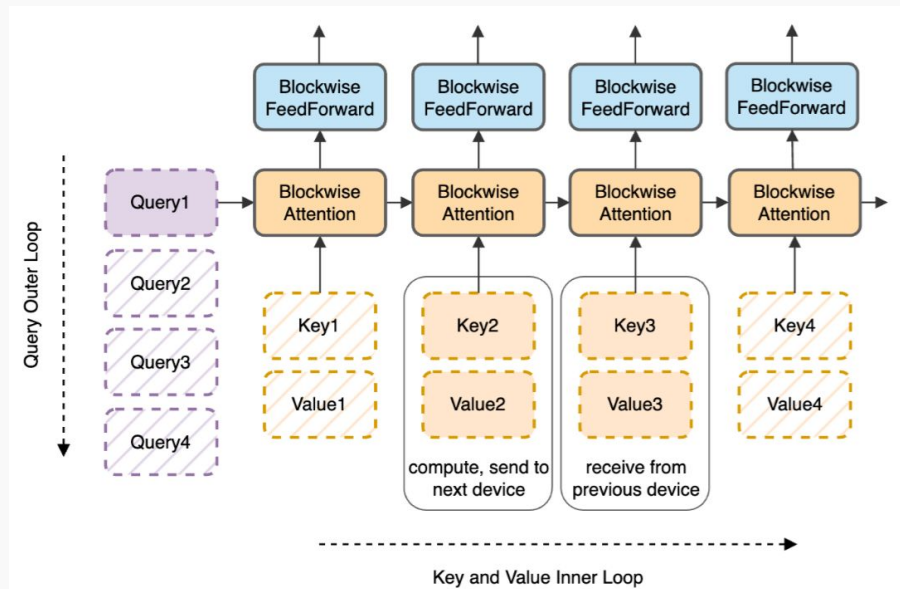
 Enable TransformerEngine CP

 Enable Torch's CP

 Enable DeepSpeed-Ulysses CP

 [PyTorch CP Post](#)

 [DS-Ulysses CP Post](#)



Possible Project Features

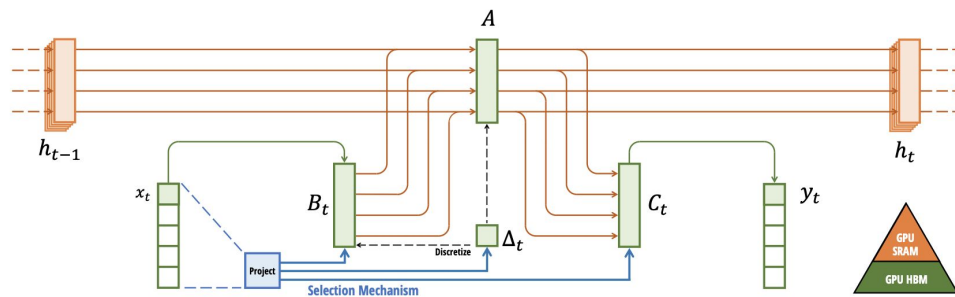
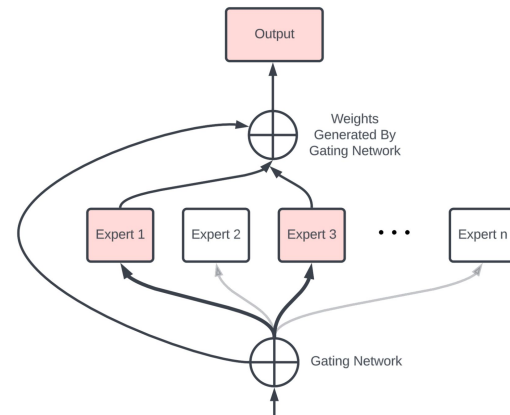
Advanced models to achieve better throughput at similar convergence.

 Enable expert parallelism (MoE)

 Benchmark throughput of different implementations for attention alternatives like the (Gated) Delta Net, Mamba, Linear Attention, xLSTM, etc.

 Hybrid models (like Qwen 3 Next)

 Quantisation aware training for better inference throughput



Now

Teams: Size of 3-4 students. Coordinate to split the work and compute.

Deliverable 1: Find a team and **submit ~1 page proposal by 14th of April** which lists the team members and roughly describes the main direction of your project **per student**, related work, milestones, etc.